

Създаване на приложение с модули. Модул NumPy.

I. Теоретична постановка

1. Създаване на приложение с модули

Модул в Python се нарича всеки файл с програма. Всеки модул може да импортира в себе си друг модул и по този начин да получи достъп до идентификаторите, които се намират в него. Така могат да се добавят например библиотеки с функции или друг код, писан от същия разработчик.

За импортиране на друг модул се използва инструкцията *import*:

```
import <име на модул>
```

Например:

```
import math
```

След това можем да се обърнем към идентификаторите от този модул така:

```
>>> x = math.pow(4,2)
```

```
>>> x
```

```
16.0
```

Имената на някои модули са много дълги и в този случай, за да направим кодът по-компактен, можем да създадем псевдоним на модула така:

```
import <модул> as <псевдоним>
```

Например:

```
import math as m
```

```
a = m.pi * 10
```

Чрез инструкция *from* могат да се импортират само определени идентификатори от даден модул:

```
from <модул> import <идентификатор> [as <псевдоним>]
```

Например:

```
from math import pi          # Импортираме само константата pi
```

При писане на собствени модули от разработчика, те обикновено са в работната папка на програмата и така могат лесно да бъдат намерени от нея.

С помощта на горните команди могат да се включват и използват не само стандартните, но и други много полезни модули – библиотеки за Python, които се разпространяват свободно. Важна особеност е, че за да бъдат намерени тези модули след тяхното инсталиране, тяхната инсталационна папка трябва да се включи в променливата на средата PYTHONPATH. Инсталирането на външни модули се прави с файловия мениджър *pip3* на Python.

Пример 1: Създайте модул с име *mod.py* , който съдържа две функции. Първата функция *say_hi()* отпечатва приветствие към потребител с въведено име, а втората функция *say_bye()* отпечатва съобщение за сбогуване. Напишете програма, която импортира модул *mod*, въвежда име на потребител от клавиатурата и за тях вика функциите от импортирания модул.

```
# модул mod.py
def say_hi(name):
    print(f"Hello, {name}!")
def say_bye(name):
    print(f"Bye, {name}!")

# код на програмата
import mod as m          # Импортиране на модул с псевдоним
st=input("Name: ")       # Въвеждане на име на потребител
m.say_hi(st)              # Викане на функция say_hi() от mod.py
m.say_bye(st)             # Викане на функция say_bye() от mod.py
```

Пример 2: Да се напише програма на Python, която по 9 въведени от потребителя цели числа създава 3x3 матрици с имена A и B. Програмата трябва да събира, да изважда и да умножава матриците A и B. Тя трябва да определя и сумата от елементите на матрица A и сумата от елементите на матрица B.

Забележка: В програмата трябва да се използва външния модул NumPy.

NumPy (Numerical Python) е свободна Python библиотека, която позволява удобна и бърза работа с многомерни масиви, извършване на операции от линейната алгебра и действия с матрици.

Инсталирането на библиотеката става в конзолен прозорец (Command Prompt за Windows) с команда:

```
pip3 install numpy
```

Импортирането на библиотеката в кода на програма става например с команда:

```
import numpy as np
```

Нужните функции от библиотеката за реализацията на поставеното задание са:

Създаване на масив

```
>>> matrix1 = np.array([1, 2, 3, 4, 5]) # Едномерен масив с пет елемента
```

Отпечатването на масив може да стане със стандартната функция print()

```
>>> print(matrix1)
```

Изваждането на матрици става с оператор -

```
>>> matrix1-matrix2
```

Събирането става с оператор +

```
>>> matrix1+matrix2
```

Умножението на матрици става с оператор @

```
>>> matrix1@matrix2
```

Определяне на сумата от елементите на матрицата

```
>>> matrix1.sum()
```

Смяна на размерността на матрицата

```
>>> matrix = matrix1.reshape(3, 3)
```

```
import numpy as np          # Импортиране на модул с псевдоним
```

```
def inputdata():            # Функция за създаване на матрици 3x3 с цели числа
```

```
    L=[]
```

```
    print("Въведете 9 цели числа: ")
```

```
    for i in range(0,9):
```

```
        while True:
```

```
            try:
```

```
                x=int(input(">>> "))
```

```
                L.append(x)
```

```
                break
```

```
            except ValueError:
```

```
                print("Нужно е цяло число!")
```

```
    matrix=np.array(L)
```

```
    matrix1=matrix.reshape(3,3)
```

```
    return matrix1
```

```
A=inputdata()
```

```
B=inputdata()
```

```
C=A-B
```

```
print("Изваждане на матриците:")
```

```
print(C)
```

```
C=A+B
```

```
print("Събиране на матриците:")
```

```
print(C)
```

```
C=A@B
```

```
print("Умножение на матриците:")
```

```
print(C)
```

```
print(f"Сумата на първата матрица е: {A.sum()}")
```

```
print(f"Сумата на втората матрица е: {B.sum()}")
```

II. Задачи за самостоятелна работа

1. Направете промени в кода на Пример 1, така че в модула mod.py да се добави още една функция, която да определя броя на символите в името на потребителя. Тази функция също да се вика в кода на програмата.

2. Направете промени в кода на Пример 2, така че програмата да записва получените при изчисленията матрици в двоичен файл.

Забележка: Използвайте следните функции от библиотеката NumPy:

Запис на масив в двоичен файл

```
>>> np.save(f, matrix)      # f е файлов указател
```

Четене на матрица от двоичен файл

```
>>> matrix = np.load(f)     # f е файлов указател
```

3. Напишете програма калкулатор за матрици с размер 2x2, която събира две матрици, изважда две матрици, умножава две матрици или умножава матрица по целочислен коефициент. Елементите на матриците да се въвеждат от клавиатурата. Потребителят да избира кое изчисление да се извърши.

Забележка: Трябва да използвате библиотеката NumPy. Умножение на матрица с константа се прави с оператор *

```
>>> 2*matrix1
```